

CoGo Feature Plan 001

Login / Register / Student Verification

Tài liệu triển khai & nghiệm thu

Phiên bản PDF được tạo từ plan/features/001-login-register.md

CoGo Feature Plan 001 — Login / Register / Student Verification

Trạng thái: PLAN ONLY — chưa được phép code trước khi plan được duyệt

Mục tiêu: dùng tài liệu này làm nguồn hướng dẫn trực tiếp cho Antigravity/AI khi triển khai phần Login–Register của CoGo.

Quy tắc dự án: mọi feature về sau phải có một file plan riêng trong `plan/features/` trước khi code. Plan phải nêu mục tiêu, phạm vi, flow, data, API, UI, business rules, acceptance criteria và cách kiểm thử.

1. Mục tiêu feature

Hoàn thiện luồng onboarding đầu tiên của CoGo theo đúng giao diện tham chiếu, nhưng chuyển từ prototype/mock hiện tại sang flow thật:

```
Đăng nhập
↓
Xác thực số điện thoại
↓
Có tài khoản? — Có —> Vào CoGo
|
Không
↓
Tạo tài khoản
↓
Xác thực số điện thoại
↓
Xác thực sinh viên
↓
Chờ/Hoàn tất xác thực
↓
Vào CoGo theo trạng thái tài khoản
```

Feature này chỉ xử lý Authentication + Registration + Student Verification onboarding. Không mở rộng sang profile, ride, payment, community hay driver onboarding trong cùng task.

2. Nguồn tham chiếu UI

Hai file tham chiếu được cung cấp:

- `Image\_1\_login\_register\_reference.jpeg`: ảnh giao diện tham khảo.
- `Image\_2\_login\_register\_reference.html`: HTML prototype, là nguồn tham chiếu chi tiết cho layout, text, màu sắc và tương tác preview.

Trong HTML tham chiếu hiện có 3 màn hình: `Đăng nhập`, `Tạo tài khoản`, `Xác thực sinh viên`. Phần preview dùng Tailwind CDN, Inter và các màu brand của CoGo. Khi chuyển sang repo React, không bê nguyên HTML preview vào production; chỉ tái sử dụng visual specification và triển khai lại bằng component/CSS của project.

Visual tokens cần giữ

```
Font: Inter
Brand dark: #0E4834
Brand sage: #6F9E87
Sage hover: #5D8B74
Badge bg: #E8F5EF
Badge text: #2D7755
Input bg: #F8FAFC
Mint: #A5DFC5 / #A2E2C8
Border: #E2E8F0
```

UI cần giữ cảm giác:

- mobile-first;
- nền sáng;
- card/form bo góc lớn;
- typography rõ;
- input có icon;
- button CoGo màu xanh sage/dark;
- segmented tab `Đăng nhập / Đăng ký`;
- trạng thái lỗi/loading rõ ràng;
- không còn theme tối của auth hiện tại.

3. Hiện trạng code cần xử lý

Frontend hiện tại

Các file chính:

```
src/user/auth/Login.tsx
src/user/auth/Register.tsx
```

```
src/user/auth/AuthLayout.tsx
src/user/auth/AuthForm.module.css
src/user/auth/AuthLayout.module.css
src/app/context/AuthContext.tsx
src/shared/lib/auth.service.ts
src/shared/lib/api.ts
src/app/routes.tsx
```

Các vấn đề phải sửa:

1. `AuthService` đang dùng `localStorage` làm mock DB. 2. Login hiện chỉ tìm user theo số điện thoại; không có xác thực danh tính thật. 3. Register đang tạo user bằng `Math.random()` và dữ liệu mock. 4. `AuthContext` khôi phục user từ `localStorage` thay vì session thật. 5. `any` đang được dùng trong catch/API/server; feature mới không được bổ sung `any`. 6. UI hiện tại dùng theme tối, không giống reference. 7. Route register hiện đi thẳng vào profile; chưa có bước xác thực sinh viên. 8. Backend `server/api/auth.ts` chỉ là skeleton. 9. `server/db/index.ts` chưa có DB client thật. 10. Migration chỉ có bảng `users` đơn giản, chưa đủ cho auth/verification.

Không xóa/refactor lan sang module khác ngoài phần auth/registration nếu không cần thiết.

4. Phạm vi màn hình

Screen A — Đăng nhập

Route: `/login`

UI theo reference:

- Logo `Cogo`.
- Badge `Đi chung an toàn`.
- Title `Chào mừng bạn!`.
- Subtitle `Đăng nhập bằng số điện thoại để tiếp tục`.
- Tab `Đăng nhập / Đăng ký`.
- Input số điện thoại.
- Button `Đăng nhập →`.
- Link/CTA tới đăng ký.
- Footer đồng ý `Điều khoản dịch vụ`.

Hành vi

1. User nhập số điện thoại. 2. Chuẩn hoá số về format thống nhất trước khi gửi backend. 3. Bấm `Đăng nhập`. 4. Backend kiểm tra account. 5. Nếu chưa có account → thông báo rõ và dẫn sang Register. 6. Nếu có account → gửi

OTP. 7. User nhập OTP ở màn hình xác thực OTP. 8. OTP đúng → tạo/khôi phục session. 9. Điều hướng theo trạng thái account:

- `student\_verified` → vào app;
- `student\_verification\_pending` → mở lại trạng thái xác thực sinh viên;
- `profile\_incomplete` → quay lại bước còn thiếu.

Không được login chỉ bằng việc biết số điện thoại. Prototype hiện tại làm vậy vì mock; production phải có bước xác thực số điện thoại.

Screen B — Tạo tài khoản

Route: `/register`

UI theo reference:

- Logo + badge.
- Title `Tạo tài khoản mới`.
- Subtitle `Kết nối và chia sẻ chuyến đi cùng sinh viên`.
- Tab `Đăng nhập / Đăng ký`.
- Họ và tên.
- Số điện thoại.
- Trường học / Giới thiệu.
- Button `Xác thực sinh viên →`.
- Footer Điều khoản dịch vụ.

Hành vi

1. Validate dữ liệu client. 2. Gửi yêu cầu đăng ký. 3. Xác minh số điện thoại bằng OTP. 4. Tạo account/profile ở trạng thái `PENDING\_STUDENT\_VERIFICATION`. 5. Chuyển tới `/register/verify-student`.

Không tạo user bằng dữ liệu giả ở frontend.

Screen C — Xác thực sinh viên

Route: `/register/verify-student`

UI theo reference:

- Progress indicator.
- Chọn trường Đại học/Cao đẳng.
- MSSV.

- Upload/chụp mặt trước thẻ sinh viên.
- Nút `Chụp trực tiếp`.
- Nút `Tải ảnh lên`.
- Badge `Cần rõ nét`.
- Nút `Gửi thông tin xác thực`.

Hành vi

1. User chọn trường. 2. Nhập MSSV. 3. Upload hoặc chụp ảnh. 4. Client kiểm tra loại file, kích thước và trạng thái ảnh. 5. Upload ảnh vào storage private. 6. Gửi metadata xác thực về backend. 7. Backend tạo submission ở trạng thái `PENDING`. 8. User thấy trạng thái `Đang chờ xác thực`. 9. Khi được duyệt/rejected, app đọc trạng thái thật từ backend và điều hướng tương ứng.

Screen D — OTP Verification

Màn hình này chưa có trong HTML tham chiếu nhưng bắt buộc về mặt flow authentication vì Login/Register hiện không có cơ chế xác thực số điện thoại thật.

UI tối giản, giữ nguyên style của reference:

Xác thực số điện thoại
Mã OTP đã gửi tới 09xx xxx xxx
[_ _ _ _ _]
Gửi lại mã (00:xx)
Xác nhận

States:

- chưa nhập đủ;
- OTP sai;
- OTP hết hạn;
- resend;
- too many attempts;
- network error;
- success.

5. Business / Auth States

Không dùng `isLoggedIn: boolean` làm business state chính.

Account status

UNVERIFIED_PHONE
PHONE_VERIFIED
PENDING_STUDENT_VERIFICATION
STUDENT_VERIFIED
STUDENT_REJECTED
SUSPENDED

Verification status

NOT_SUBMITTED
PENDING
APPROVED
REJECTED

Quy tắc điều hướng

Không có session
→ /login
Có session + PENDING_STUDENT_VERIFICATION
→ /register/verify-student
Có session + STUDENT_REJECTED
→ /register/verify-student + lý do
Có session + STUDENT_VERIFIED
→ app home
SUSPENDED
→ màn hình account bị tạm khóa

Backend phải là nơi quyết định status cuối cùng. Frontend chỉ hiển thị và điều hướng theo response.

6. Data model cần có

`users`

Giữ bảng user hiện có nhưng phải mở rộng tối thiểu:

id
name
phone
email (nullable nếu chưa cần)

```
role
status
created_at
updated_at
```

Không lưu password/OTP plaintext trong `users` nếu dùng provider auth.

`student\_profiles`

```
user_id
university_id
student_id
verification_status
verified_at
rejection_reason
created_at
updated_at
```

`student\_verifications`

```
id
user_id
university_id
student_id
card_front_path
status
submitted_at
reviewed_at
reviewer_id
rejection_reason
created_at
updated_at
```

`universities`

```
id
name
code
status
student_id_rule (optional)
```

```
created_at
updated_at
```

Mục tiêu là không hardcode danh sách trường trong component lâu dài.

7. Storage cho ảnh thẻ sinh viên

Ảnh thẻ sinh viên là dữ liệu nhạy cảm.

Yêu cầu:

- dùng Supabase Storage/private bucket hoặc storage tương đương;
- không dùng public URL cho ảnh thẻ;
- frontend upload qua flow được kiểm soát;
- backend kiểm tra ownership/quyền truy cập;
- nếu cần reviewer/admin xem ảnh thì dùng signed URL có thời hạn;
- validate MIME type + file size;
- không tin extension do client gửi.

Client UX có thể hiển thị preview, nhưng preview không đồng nghĩa ảnh đã được upload thành công.

8. Auth architecture đề xuất

Vì repo đã có Supabase và backend Node, triển khai theo hướng:

```
React
↓
Auth service
↓
Supabase Auth / phone OTP
↓
Session / JWT
↓
Backend middleware
↓
User profile + student verification DB
```

Backend phải verify session/JWT trước khi xử lý các endpoint cần đăng nhập.

Không tiếp tục dùng:

```
localStorage = source of truth
```

Có thể dùng browser storage cho persistence của session do auth provider quản lý, nhưng không tự lưu object `User` như cơ chế authentication.

9. API contract

Auth

```
POST /api/auth/otp/request
```

```
POST /api/auth/otp/verify
```

```
GET /api/auth/me
```

```
POST /api/auth/logout
```

Registration

```
POST /api/auth/register/profile
```

Payload dự kiến:

```
{
  "name": "Nguyễn Văn A",
  "phone": "09xxxxxxxx",
  "universityId": "..."
}
```

Student verification

```
POST /api/student-verifications
```

```
GET /api/student-verifications/me
```

Không cho frontend tự truyền:

```
role
status
verified_at
reviewer_id
```

Đây là server-owned fields.

10. Validation rules

Phone

- Chỉ nhận số điện thoại Việt Nam theo format mà business chốt.
- Normalize về một format duy nhất trước khi lưu.
- Không dùng `length >= 10` đơn thuần như code hiện tại.
- Hiện thị lỗi ngay tại field.

Name

- Trim khoảng trắng.
- Không cho chuỗi rỗng.
- Có giới hạn độ dài hợp lý.

University

- Bắt buộc chọn từ danh sách hợp lệ.
- Không tin `universityId` không tồn tại.

MSSV

- Không hardcode một regex chung cho mọi trường nếu chưa có rule chính thức.
- Basic validation ở client; validation business ở server.

Student card image

- Bắt buộc.
- Chỉ chấp nhận MIME type cho phép.
- Giới hạn dung lượng.
- Từ chối file lỗi/không hợp lệ.

11. UI states bắt buộc

Mỗi màn hình phải xử lý ít nhất:

Default
Focus
Disabled
Loading
Success
Validation error
Server error
Network error
Empty/Incomplete

Đặc biệt:

Login

- Số điện thoại sai.
- Account không tồn tại.
- OTP sai/hết hạn.
- Resend OTP.
- Login thành công.

Register

- SĐT đã tồn tại.
- Tên không hợp lệ.
- Trường chưa chọn.
- OTP fail.
- Network error.

Student verification

- MSSV thiếu.
- Trường thiếu.
- Ảnh thiếu.
- File không hợp lệ.
- Upload fail.
- Submit success.
- Submission đang pending.
- Rejected + reason.

12. Component structure dự kiến

Không tạo folder mới nếu không cần; bám cấu trúc hiện tại.

```
src/user/auth/  
├── AuthLayout.tsx  
├── AuthLayout.module.css  
├── Login.tsx  
├── Register.tsx  
├── OtpVerification.tsx  
├── StudentVerification.tsx  
├── AuthForm.module.css  
├── verification/  
├── StudentCardUpload.tsx  
└── UniversitySelect.tsx
```

Shared:

```
src/shared/lib/  
├── api.ts  
└── auth.service.ts
```

Context:

```
src/app/context/AuthContext.tsx
```

Backend:

```
server/api/auth.ts  
server/api/student_verifications.ts  
server/db/index.ts  
server/shared/middleware.ts
```

Types:

```
shared/types/user.ts  
shared/types/auth.ts  
shared/types/studentVerification.ts
```

Migration:

13. Route design

```
/login
/register
/register/verify-phone
/register/verify-student
```

Protected routes khác phải đi qua `ProtectedRoute` và đọc session thật.

Không cho user truy cập trực tiếp `/register/verify-student` nếu chưa có authenticated session/registration state phù hợp.

14. Authentication sequence

Login

```
User
↓
POST /auth/otp/request
↓
OTP Provider
↓
User nhập OTP
↓
POST /auth/otp/verify
↓
Session/JWT
↓
GET /auth/me
↓
Router theo account status
```

Register

Register form

↓
Validate client
↓
OTP verification
↓
Create/update user profile
↓
PENDING_STUDENT_VERIFICATION
↓
Student verification form
↓
Upload card
↓
Submit verification
↓
PENDING

15. Security requirements

- Không lưu OTP plaintext.
- OTP có expiry và giới hạn số lần thử.
- Resend có cooldown/rate limit.
- Login/OTP endpoint có rate limit.
- Không trả lỗi làm lộ quá nhiều thông tin tài khoản.
- Backend kiểm tra session/authorization.
- Người dùng chỉ xem submission của chính mình.
- Ảnh thẻ sinh viên private.
- Audit các hành động review/reject/approve.
- Không tin role/status do client gửi lên.
- Không dùng `Math.random()` để tạo identity chính thức.

16. CI/CD và cách verify

Workflow hiện tại của repo có build/lint/dry-run/deploy preview. Feature này phải giữ cho pipeline chạy được.

Trước merge:

```
npm ci
npm run build
```

Nếu project có script lint riêng thì bổ sung/chạy đúng script hiện hành; không đổi pipeline chỉ để bỏ qua lỗi.

PR phải có:

- preview environment;
- check `/api/health`;
- test login/register flow;
- không commit secret;
- không commit ảnh thẻ thật.

17. Test cases bắt buộc

Login

TC-LOGIN-001: số điện thoại hợp lệ
TC-LOGIN-002: số điện thoại sai format
TC-LOGIN-003: account không tồn tại
TC-LOGIN-004: OTP đúng
TC-LOGIN-005: OTP sai
TC-LOGIN-006: OTP hết hạn
TC-LOGIN-007: resend OTP
TC-LOGIN-008: quá số lần thử
TC-LOGIN-009: network error
TC-LOGIN-010: session restore

Register

TC-REG-001: đăng ký hợp lệ
TC-REG-002: SĐT đã tồn tại
TC-REG-003: tên rỗng
TC-REG-004: trường chưa chọn
TC-REG-005: OTP fail
TC-REG-006: tạo profile thành công

Student verification

TC-STU-001: chọn trường
TC-STU-002: MSSV rỗng

TC-STU-003: file sai loại
TC-STU-004: file quá lớn
TC-STU-005: upload success
TC-STU-006: upload fail
TC-STU-007: submit success
TC-STU-008: pending status
TC-STU-009: rejected status
TC-STU-010: user chỉ xem được submission của mình

18. Acceptance Criteria — chỉ coi feature DONE khi đủ tất cả

UI

- ☐ Login giống visual reference ở mức component/layout/spacing/typography.
- ☐ Register giống visual reference.
- ☐ Student verification giống visual reference.
- ☐ Có OTP screen bổ sung nhưng cùng design system.
- ☐ Responsive mobile/desktop.
- ☐ Không còn auth dark theme cũ.

Functional

- ☐ Login không còn mock localStorage.
- ☐ Register không còn mock user.
- ☐ OTP flow hoạt động thật.
- ☐ Session hoạt động thật.
- ☐ Student verification hoạt động thật.
- ☐ Protected route hoạt động theo session.
- ☐ Account status điều hướng đúng.

Backend / DB

- ☐ API auth thật.
- ☐ DB schema đủ.
- ☐ Migration có thể chạy từ môi trường sạch.

- ☐ Authorization ở backend.
- ☐ Student verification status do server kiểm soát.

Security

- ☐ OTP rate limit.
- ☐ Session/JWT verify.
- ☐ Private student-card storage.
- ☐ Không secret trong client.
- ☐ Không `Math.random()` cho identity.
- ☐ Không dùng `any` trong code mới.

QA

- ☐ Test cases critical pass.
- ☐ Build pass.
- ☐ Preview deploy pass.
- ☐ Có evidence/test note trong PR.

19. Definition of Done

Feature Login–Register chỉ được merge vào `main` khi:

```
UI
+ Auth
+ Registration
+ OTP
+ Student Verification
+ DB
+ API
+ Security
+ Error states
+ QA
+ CI/CD
```

đều đã pass acceptance criteria.

Không coi việc “UI bấm được” là hoàn thành feature.

20. Thứ tự implementation bắt buộc

Antigravity triển khai theo đúng thứ tự, không nhảy bước:

01. Cập nhật shared types
02. Tạo DB migration
03. Cấu hình DB/Auth provider
04. Implement backend auth + middleware
05. Implement OTP flow
06. Implement registration profile
07. Implement student verification + storage
08. Cập nhật AuthContext/session handling
09. Cập nhật routes/guards
10. Rebuild Login/Register UI theo reference
11. Rebuild Student Verification UI theo reference
12. Add loading/error/empty states
13. Add tests
14. Build + deploy preview
15. QA toàn bộ acceptance criteria

21. Không làm trong feature này

Không tự ý thêm:

- payment;
- wallet;
- ride matching;
- chat/community;
- AI;
- driver onboarding;
- social feed;
- recommendation;
- microservice;
- redesign toàn bộ app.

Các phần này phải có plan riêng trước khi implement.

22. Quy tắc Plan cho toàn bộ dự án về sau

Mỗi feature mới phải tạo:

```
plan/features/002-<feature-name>.md
plan/features/003-<feature-name>.md
...
```

Mỗi plan tối thiểu có:

1. Mục tiêu
2. Phạm vi / ngoài phạm vi
3. UI/UX
4. Business rules
5. Flow / state machine
6. Data model
7. API contract
8. Security
9. Test cases
10. Acceptance criteria
11. Definition of Done
12. Implementation order

AI không được code feature khi chưa đọc plan tương ứng.

23. Kết quả cần đạt sau feature 001

Khi hoàn tất, CoGo phải chạy được flow thật:

```
Mở app
↓
Đăng nhập / Đăng ký
↓
Xác thực số điện thoại
↓
Tạo profile
↓
Xác thực sinh viên
```

↓

Theo dõi trạng thái xác thực

↓

Vào app đúng theo account status

Và quan trọng nhất:

Login/Register phải trở thành nền auth thật của CoGo, không còn là màn hình demo nổi bằng `localStorage` và mock user.